

HDOC Day-39 — Complete Solutions

1. Variadic Function: Weighted Average

The following C program uses **stdarg.h** to accept a count followed by value-weight pairs. It validates that every weight is non-negative and also handles the case where the total weight is zero.

```
#include <stdio.h>
#include <stdarg.h>

double weightedAverage(int count, ...)
{
    va_list args;
    double weightedSum = 0.0;
    double totalWeight = 0.0;

    if (count <= 0) {
        printf("Error: count must be positive.\n");
        return 0.0;
    }

    va_start(args, count);

    for (int i = 0; i < count; i++) {
        double value = va_arg(args, double);
        double weight = va_arg(args, double);

        if (weight < 0.0) {
            printf("Error: weight %d is negative.\n", i + 1);
            va_end(args);
            return 0.0;
        }

        weightedSum += value * weight;
        totalWeight += weight;
    }

    va_end(args);

    if (totalWeight == 0.0) {
        printf("Error: total weight is zero.\n");
        return 0.0;
    }

    return weightedSum / totalWeight;
}

int main(void)
{
    double result;

    result = weightedAverage(3,
                             80.0, 0.3,
                             75.0, 0.2,
                             90.0, 0.5);

    printf("Weighted Average: %.2f\n", result);

    return 0;
}
```

Output: Weighted Average: 84.00

Calculation: $(80 \times 0.3 + 75 \times 0.2 + 90 \times 0.5) / (0.3 + 0.2 + 0.5) = 84.00$

2. Command-Line Integer Processing

This program reads command-line arguments, ignores non-integer arguments, sorts valid integers in ascending order, and calculates the minimum, maximum, average, median, and second-largest distinct value. It uses separate user-defined functions for validation, sorting, and calculations.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h>
#include <errno.h>
#include <ctype.h>

int isInteger(const char *str, int *value)
{
    char *end;
    long num;

    if (str == NULL || *str == '\\0')
        return 0;

    errno = 0;
    num = strtol(str, &end, 10);

    if (errno != 0 || *end != '\\0')
        return 0;

    if (num < INT_MIN || num > INT_MAX)
        return 0;

    *value = (int)num;
    return 1;
}

void sortAscending(int a[], int n)
{
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                int temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}

int minimum(int a[], int n)
{
    int min = a[0];

    for (int i = 1; i < n; i++)
        if (a[i] < min)
            min = a[i];

    return min;
}

int maximum(int a[], int n)
{
    int max = a[0];

    for (int i = 1; i < n; i++)
        if (a[i] > max)
            max = a[i];

    return max;
}

double average(int a[], int n)
{
    long long sum = 0;
```

```

        for (int i = 0; i < n; i++)
            sum += a[i];

        return (double)sum / n;
}

double median(int a[], int n)
{
    if (n % 2 != 0)
        return a[n / 2];

    return (a[n / 2 - 1] + (double)a[n / 2]) / 2.0;
}

int secondLargestDistinct(int a[], int n, int *result)
{
    for (int i = n - 2; i >= 0; i--) {
        if (a[i] != a[n - 1]) {
            *result = a[i];
            return 1;
        }
    }

    return 0;
}

int main(int argc, char *argv[])
{
    int values[argc > 1 ? argc - 1 : 1];
    int n = 0;
    int value;
    int second;

    for (int i = 1; i < argc; i++) {
        if (isInteger(argv[i], &value)) {
            values[n++] = value;
        } else {
            printf("Invalid argument ignored: %s\n", argv[i]);
        }
    }

    if (n == 0) {
        printf("No valid integers were provided.\n");
        return 0;
    }

    sortAscending(values, n);

    printf("Valid integers in ascending order:");
    for (int i = 0; i < n; i++)
        printf(" %d", values[i]);
    printf("\n");

    printf("Minimum: %d\n", minimum(values, n));
    printf("Maximum: %d\n", maximum(values, n));
    printf("Average: %.2f\n", average(values, n));
    printf("Median: %.2f\n", median(values, n));

    if (secondLargestDistinct(values, n, &second))
        printf("Second-largest distinct value: %d\n", second);
    else
        printf("Second-largest distinct value: Does not exist\n");

    return 0;
}

```

Compilation and Execution

Save the source code as **program.c**, then compile it using:

```
gcc program.c -o program
```

Run the program with the command-line arguments:

```
./program 12 5 abc 8 12 20 3
```

Sample Output

```
Invalid argument ignored: abc
Valid integers in ascending order: 3 5 8 12 12 20
Minimum: 3
Maximum: 20
Average: 10.00
Median: 10.00
Second-largest distinct value: 12
```

Note: The program also handles cases with no valid integers and cases where a second-largest distinct value does not exist.